

Weak Scaling of NVSHMEM Applied to Hashed Distributed Structured Data

Andrew Davis, Hans Johansen, Xinfeng Gao, Stephen Guzik

CFD & Propulsion Laboratory
Mechanical & Aerospace Engineering
University of Virginia

November 16, 2025

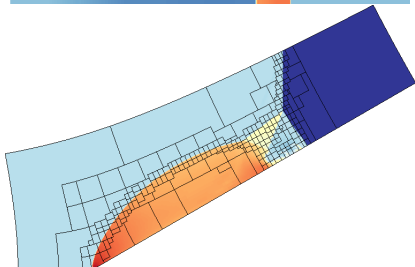


Key take-away: We found NVSHMEM outperforms Cuda-aware MPI by 64-84% when using the same algorithm and hardware

- Test problem: Compressible Euler equations with a 4th-order (space/time) finite volume (FV) discretization
- Application was written with the HashBrick library
 - Using NVSHMEM: Stagger communication of data in GPU kernels
 - Comparing: MPI with bulk synchronizations V. point-to-point NVSHMEM
 - Application is latency bound
- Evaluated the weak scaling and performance of the application

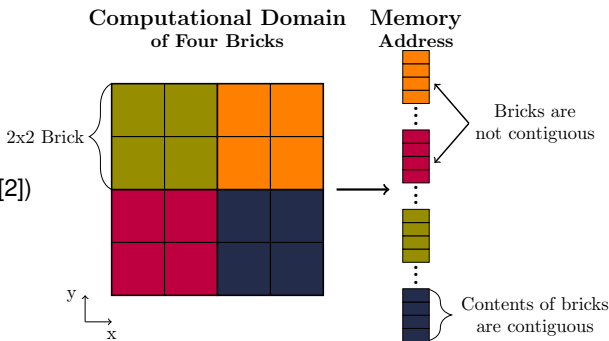
The HashBrick library was developed for massively parallel, structured grid, computations on GPUs or CPUs

- Data structures:
 - Bricks -> **Data locality**
- Communication:
 - SHMEM -> **Overlap of computation and communication**
- Caching:
 - Shared memory -> **Spatial locality**
- Avoided:
 - Large arrays of multidimensional data
 - Bulk synchronizations



Bricks are a type of domain decomposition that store data contiguously in memory and work very well for structured data on GPU (Lakshminarasimhan et al. 2024 [1])

- Easy indexing of data
- Provides performance portability
 - (Antepara et al. 2023 [2])
- Cache reuse and vectorization
 - (Zhao et al. 2019 [3])

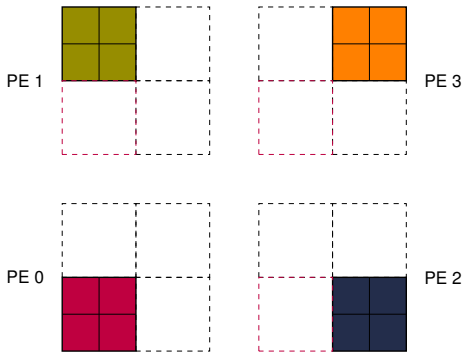


- HashBrick uses the OpenSHMEM standard to communicate data to other processing elements (PEs)

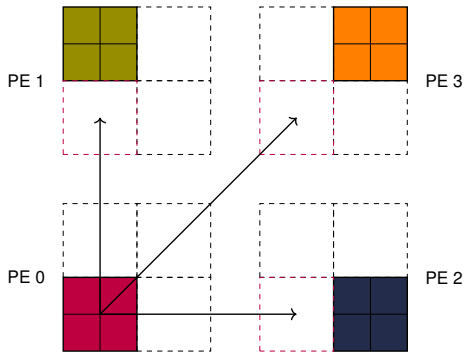
- GPU: NVSHMEM library
- CPU: OpenMPI (OpenSHMEM implementation)

- Primary advantage:

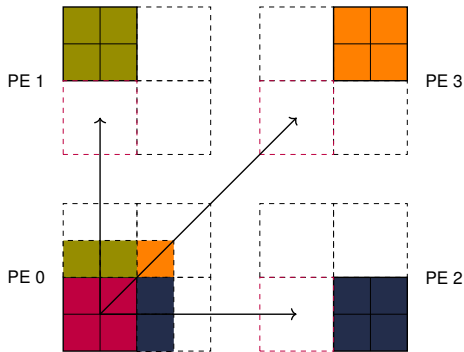
- Stagger communication
- No bulk synchronizations in the algorithm
- No packing or unpacking data



- HashBrick uses the OpenSHMEM standard to communicate data to other processing elements (PEs)
 - GPU: NVSHMEM library
 - CPU: OpenMPI (OpenSHMEM implementation)
- Primary advantage:
 - Stagger communication
 - No bulk synchronizations in the algorithm
 - No packing or unpacking data



- HashBrick uses the OpenSHMEM standard to communicate data to other processing elements (PEs)
 - GPU: NVSHMEM library
 - CPU: OpenMPI (OpenSHMEM implementation)
- Primary advantage:
 - Stagger communication
 - No bulk synchronizations in the algorithm
 - No packing or unpacking data



All data structures are created on the CPU and must be “static” before they are used on the GPU

- HashLayouts:

User Perspective

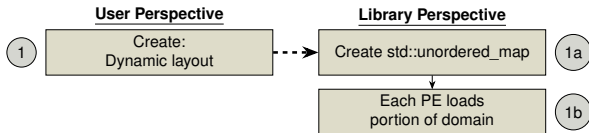
Library Perspective

- Store meta-data in hash tables
- Define communication patterns
- Iteration schedules

All data structures are created on the CPU and must be “static” before they are used on the GPU

- HashLayouts:

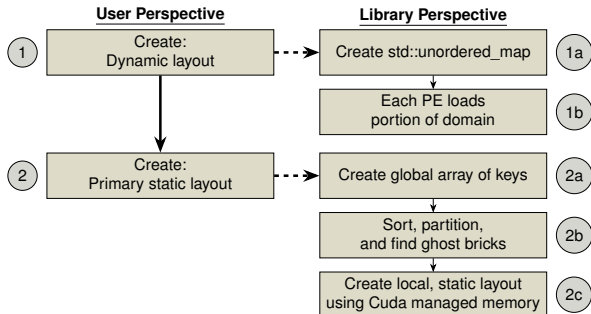
- Store meta-data in hash tables
- Define communication patterns
- Iteration schedules



All data structures are created on the CPU and must be “static” before they are used on the GPU

● HashLayouts:

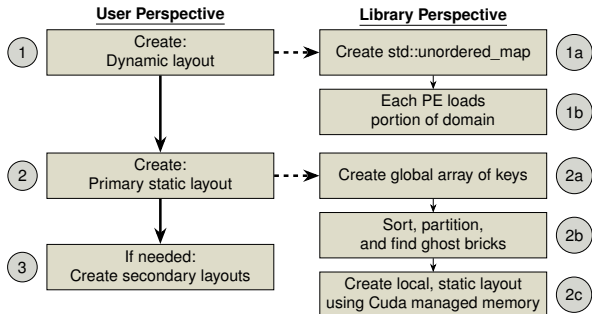
- Store meta-data in hash tables
- Define communication patterns
- Iteration schedules



All data structures are created on the CPU and must be “static” before they are used on the GPU

● HashLayouts:

- Store meta-data in hash tables
- Define communication patterns
- Iteration schedules



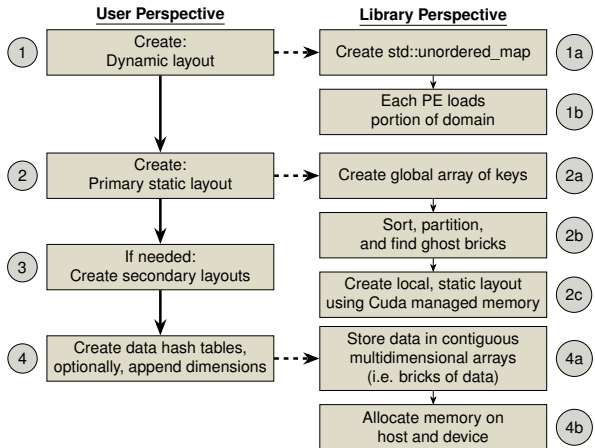
All data structures are created on the CPU and must be “static” before they are used on the GPU

● HashLayouts:

- Store meta-data in hash tables
- Define communication patterns
- Iteration schedules

● HashData:

- Store data in HashGraph (Green 2021 [4])



Compressible Euler Equations

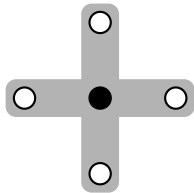
Algorithm



$$\frac{d}{dt} \mathbf{U}_i = \frac{-1}{\Delta x} \sum_{d=0}^{D-1} \left(\mathbf{F}_{i+\frac{1}{2}} \mathbf{e}^d - \mathbf{F}_{i-\frac{1}{2}} \mathbf{e}^d \right)$$

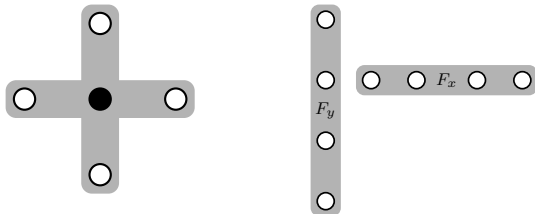
$$\frac{d}{dt} \mathbf{U}_i = \frac{-1}{\Delta x} \sum_{d=0}^{D-1} \left(\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d} - \mathbf{F}_{i-\frac{1}{2}\mathbf{e}^d} \right) \quad \mathbf{W}_i = \mathcal{F}(\mathbf{U}_i) ,$$

- Step 1: Calculate $\mathbf{W}_i$ -> Exchange $\mathbf{W}_i$



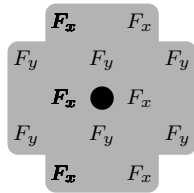
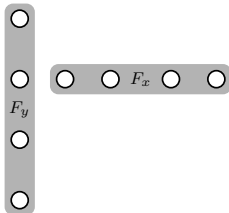
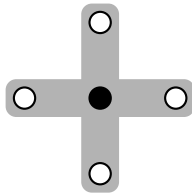
$$\frac{d}{dt} \mathbf{U}_i = \frac{-1}{\Delta x} \sum_{d=0}^{D-1} \left(\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d} - \mathbf{F}_{i-\frac{1}{2}\mathbf{e}^d} \right) \quad \mathbf{W}_i = \mathcal{F}(\mathbf{U}_i) ,$$

- Step 1: Calculate $\mathbf{W}_i$ -> Exchange $\mathbf{W}_i$
- Step 2: Calculate $\mathbf{W}_{i+\frac{1}{2}\mathbf{e}^d}$ -> Exchange $\mathbf{W}_{i+\frac{1}{2}\mathbf{e}^d}$



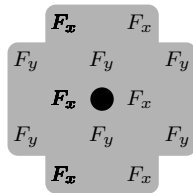
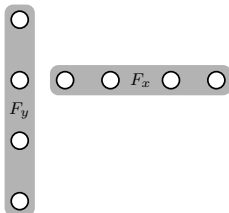
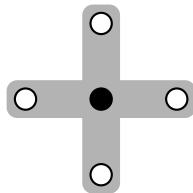
$$\frac{d}{dt} \mathbf{U}_i = \frac{-1}{\Delta x} \sum_{d=0}^{D-1} \left(\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d} - \mathbf{F}_{i-\frac{1}{2}\mathbf{e}^d} \right) \quad \mathbf{W}_i = \mathcal{F}(\mathbf{U}_i), \quad \mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d} = \mathcal{F}(\mathbf{W}_i)$$

- Step 1: Calculate $\mathbf{W}_i$ -> Exchange $\mathbf{W}_i$
- Step 2: Calculate $\mathbf{W}_{i+\frac{1}{2}\mathbf{e}^d}$ -> Exchange $\mathbf{W}_{i+\frac{1}{2}\mathbf{e}^d}$
- Step 3: Calculate $\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d}$ -> Exchange $\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d}$



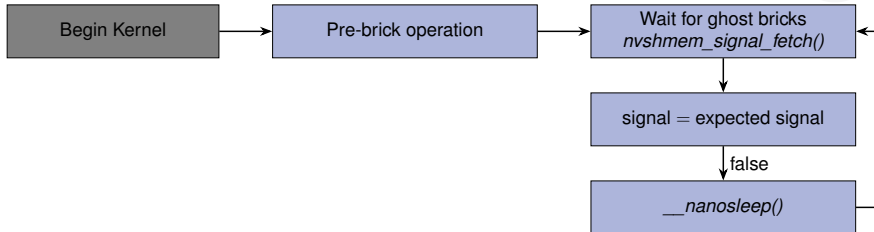
$$\frac{d}{dt} \mathbf{U}_i = \frac{-1}{\Delta x} \sum_{d=0}^{D-1} \left(\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d} - \mathbf{F}_{i-\frac{1}{2}\mathbf{e}^d} \right) \quad \mathbf{W}_i = \mathcal{F}(\mathbf{U}_i), \quad \mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d} = \mathcal{F}(\mathbf{W}_i)$$

- Step 1: Calculate $\mathbf{W}_i$ -> Exchange $\mathbf{W}_i$
- Step 2: Calculate $\mathbf{W}_{i+\frac{1}{2}\mathbf{e}^d}$ -> Exchange $\mathbf{W}_{i+\frac{1}{2}\mathbf{e}^d}$
- Step 3: Calculate $\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d}$ -> Exchange $\mathbf{F}_{i+\frac{1}{2}\mathbf{e}^d}$
- Step 4: Update $\mathbf{U}_i$ -> Exchange $\mathbf{U}_i$

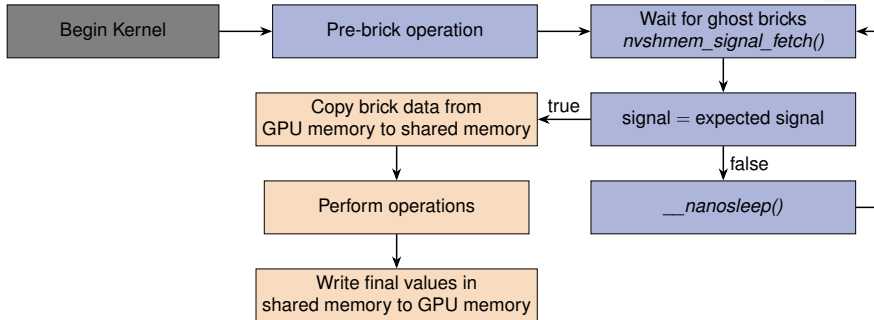


Begin Kernel

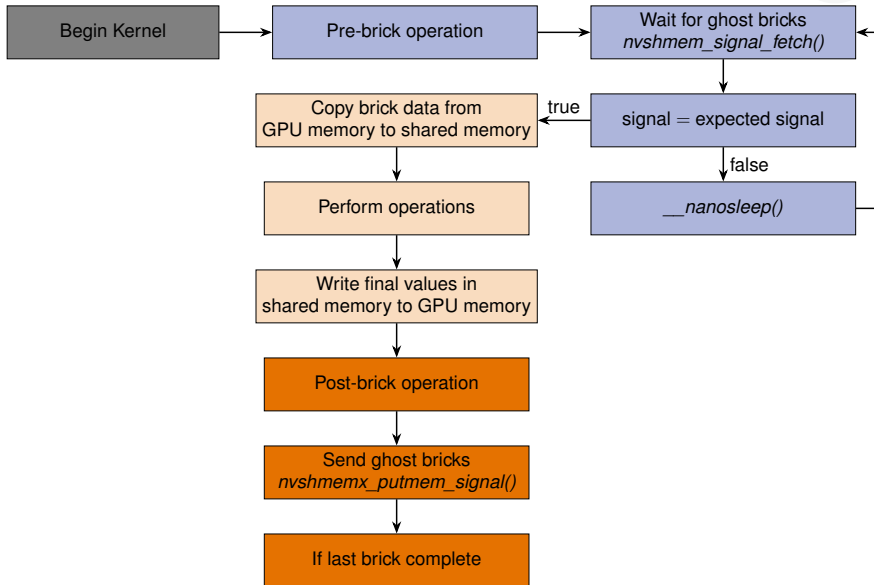
Software approach - Communication



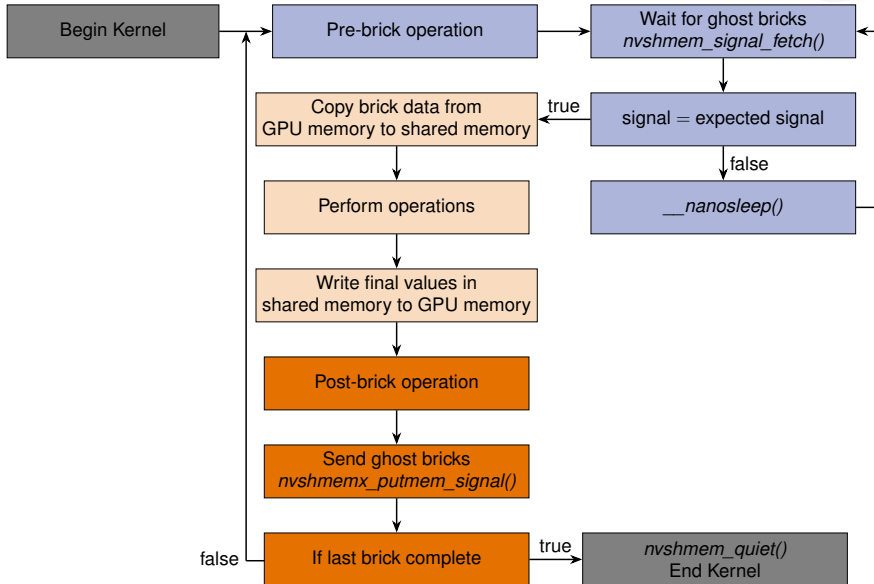
Software approach - Communication



Software approach - Communication

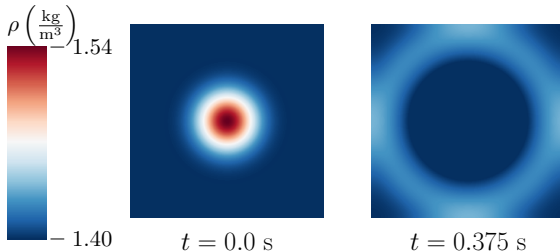


Software approach - Communication



Problem setup - Gaussian pulse

Stress test communication: Our algorithm uses a minimal amount of operations for a 4th-order accurate solution



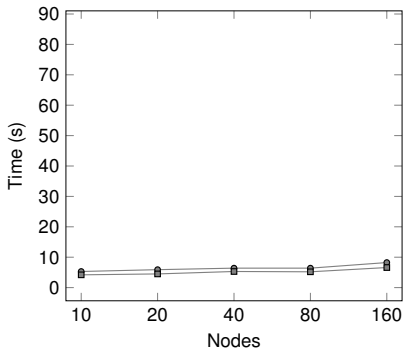
- 1 Node = 1 NVIDIA GH200 Superchip (Vista)

Number of Nodes	Domain Size	
	<i>3D HashBrick Application</i>	<i>2D Jacobi Benchmark</i>
10	$255 \times 255 \times 255$	8192×8192
20	$323 \times 323 \times 323$	11584×11584
40	$407 \times 407 \times 407$	16384×16384
80	$511 \times 511 \times 511$	23170×23170
160	$643 \times 643 \times 643$	32768×32768

- 83 million to 1.3 billion degrees of freedom

NVSHMEM outperforms GPU-aware MPI by 64%-84%

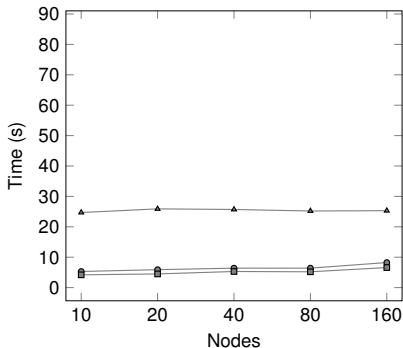
- Setup:
 - Creation of data structure



—■— Setup (MPI)
—●— Setup (NVSHMEM)

NVSHMEM outperforms GPU-aware MPI by 64%-84%

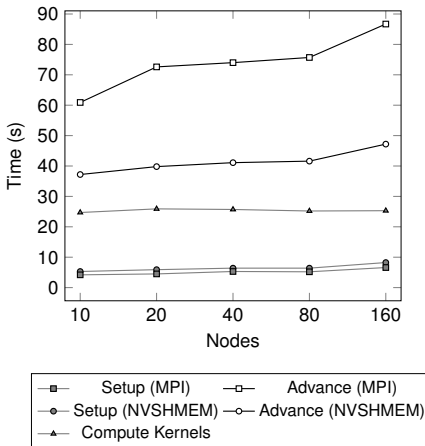
- Setup:
 - Creation of data structure
- Compute Kernels:
 - computations without data exchanges



—■— Setup (MPI)
—●— Setup (NVSHMEM)
—▲— Compute Kernels

NVSHMEM outperforms GPU-aware MPI by 64%-84%

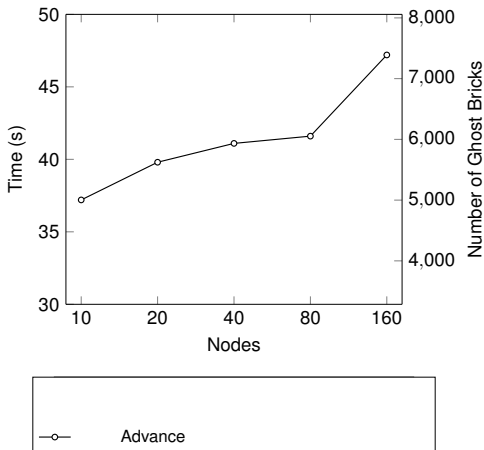
- Setup:
 - Creation of data structure
- Compute Kernels:
 - computations without data exchanges
- Advance:
 - Computations and data exchanges



A better partition of the domain and a decrease in the number of ghost bricks results in lower wall clock time and more efficient scaling

● Advance

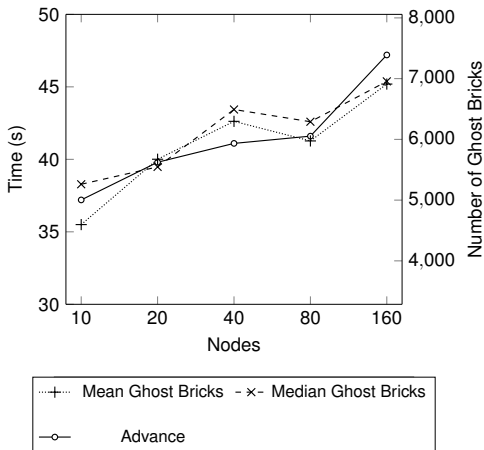
- Morton curve
- Optimizes bricks per PE



A better partition of the domain and a decrease in the number of ghost bricks results in lower wall clock time and more efficient scaling

● Advance

- Morton curve
- Optimizes bricks per PE



Results - Weak scaling optimizations

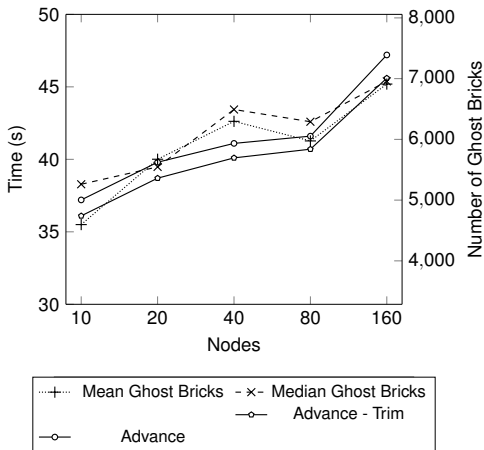
A better partition of the domain and a decrease in the number of ghost bricks results in lower wall clock time and more efficient scaling

- Advance

- Morton curve
- Optimizes bricks per PE

- Advance - trim

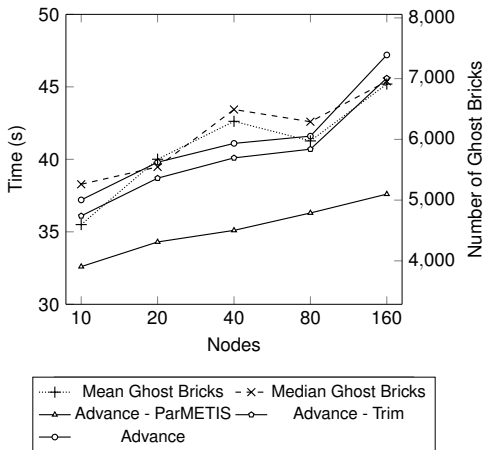
- Morton curve with “trimmed” bricks



Results - Weak scaling optimizations

A better partition of the domain and a decrease in the number of ghost bricks results in lower wall clock time and more efficient scaling

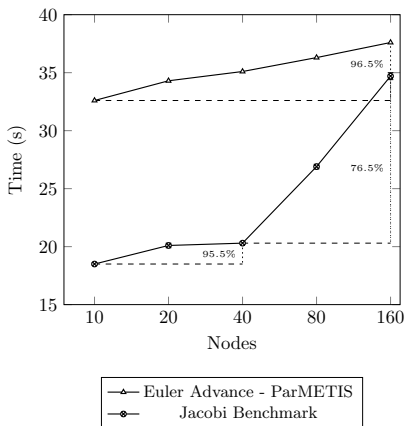
- Advance
 - Morton curve
 - Optimizes bricks per PE
- Advance - trim
 - Morton curve with “trimmed” bricks
- Advance - ParMETIS
 - ParMETIS graph partitioning with trim
 - Highly optimized edge cuts across faces



HashBrick and a NVIDIA benchmark exhibit similar scaling from 10-40 nodes

$$\text{efficiency} = 100\% \times \left((t_n/t_1)^{\frac{1}{(1-n)}} \right)$$

- efficiency is measuring an average loss per doubling
- Jacobi Benchmark
 - Modified NVIDIA strong-scaling benchmark
 - “Sanity check” for similar behavior



- NVSHMEM outperforms GPU-aware MPI by 64-84%
- Both approaches exhibit similar scaling
- Optimizations to data communication and partitioning have a significant impact on performance
- Future work:
 - Addition of adaptive mesh refinement to test HashBrick's sparse data capabilities
 - Addition of more complex features to the algorithm, such as a limiter, Riemann solver, viscous terms, etc.

- Work at UVA was supported by the National Science Foundation under Grant Number 2403579
- Dr. Johansen was supported in part by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Base Math and SciDAC / FASTMath programs under contract number DE-AC02-05CH11231
- The authors acknowledge the Texas Advanced Computing Center (TACC) at The University of Texas at Austin for providing computational resources that have contributed to the research results reported within this paper. URL: <http://www.tacc.utexas.edu>
- The authors acknowledge Research Computing (RC) at The University of Virginia for providing computational resources and technical support that have contributed to the research results reported within this paper. URL: <https://www.rc.virginia.edu/>

- [1] Mahesh Lakshminarasimhan et al. “Bricks: A high-performance portability layer for computations on block-structured grids”. In: *The International Journal of High Performance Computing Applications* 38.6 (2024), pp. 549–567. DOI: 10.1177/10943420241268288. eprint: <https://doi.org/10.1177/10943420241268288>. URL: <https://doi.org/10.1177/10943420241268288>.
- [2] Oscar Antepara et al. “Performance Portability Evaluation of Blocked Stencil Computations on GPUs”. In: *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. SC-W '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1007–1018. ISBN: 9798400707858. DOI: 10.1145/3624062.3624177. URL: <https://doi.org/10.1145/3624062.3624177>.

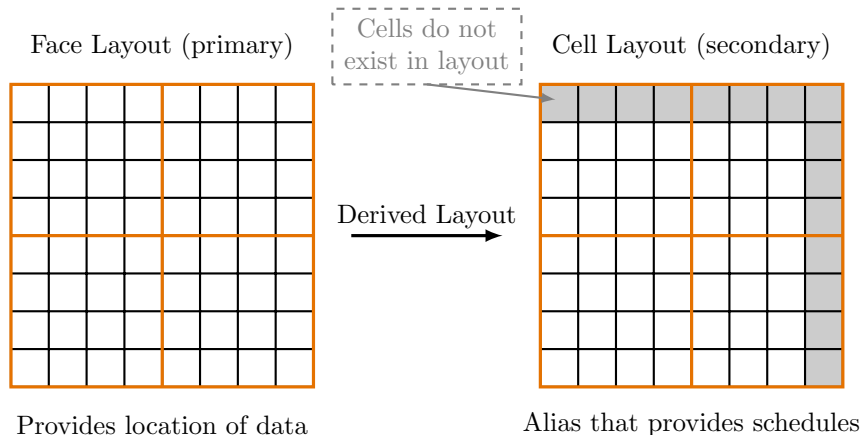
- [3] Tuowen Zhao et al. “Exploiting reuse and vectorization in blocked stencil computations on CPUs and GPUs”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '19. Denver, Colorado: Association for Computing Machinery, 2019. ISBN: 9781450362290. DOI: 10.1145/3295500.3356210. URL: <https://doi.org/10.1145/3295500.3356210>.
- [4] Oded Green. “HashGraph—Scalable Hash Tables Using a Sparse Graph Data Structure”. In: *ACM Trans. Parallel Comput.* 8.2 (July 2021). ISSN: 2329-4949. DOI: 10.1145/3460872. URL: <https://doi.org/10.1145/3460872>.

Thank you for your attention!
Questions are welcome!



Key data structures - HashLayout

HashLayout provides: the logic to iterate over data, the communication patterns for data, and are VERY lightweight

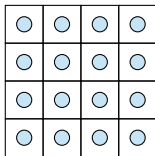


HashData stores bricks of multidimensional arrays in a hash table (CPU) or HashGraph (GPU) (Green 2021)

- Enable a flexible index space with C++ template parameter pack

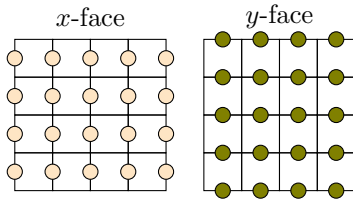
Cell-centered data

$U(\text{cell-index, component})$



Face-centered data

$F(\text{face-index, direction, component})$

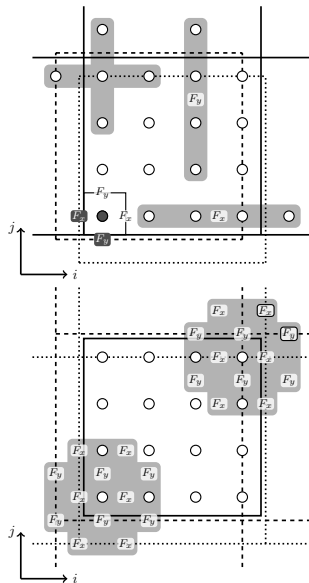


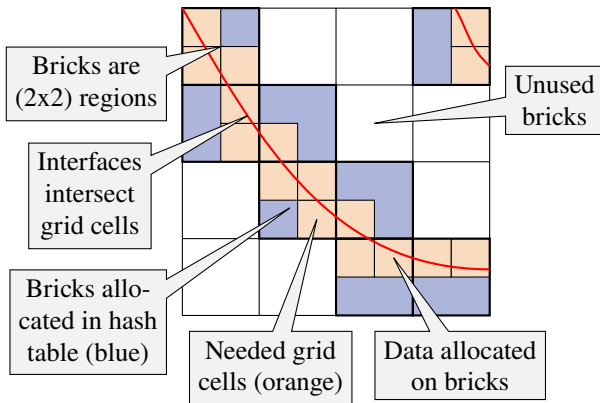
Compressible Euler Equations

Algorithm

```

for  $n \leftarrow 0$  to time steps do
  for  $s \leftarrow 0$  to RK4 stages do
    // Step 1: Compute primitive variables
    for each Brick in Domain do
      WaitForGhostBricks( $\langle \mathbf{U} \rangle_i^s$ )
       $\langle \mathbf{W} \rangle_i \leftarrow \text{Primitive}(\langle \mathbf{U} \rangle_i^s)$ 
      SendGhostBricks( $\langle \mathbf{W} \rangle_i$ )
    end for
    // Step 2: Calculate primitive face-averaged values
    for each Brick in Domain do
      WaitForGhostBricks( $\langle \mathbf{W} \rangle_i$ )
       $\langle \mathbf{W} \rangle_{i+\frac{1}{2}\mathbf{e}^d} \leftarrow \text{FaceValues}(\langle \mathbf{W} \rangle_i)$ 
      SendGhostBricks( $\langle \mathbf{W} \rangle_{i+\frac{1}{2}\mathbf{e}^d}$ )
    end for
    // Step 3: Calculate flux
    for each Brick in Domain do
      WaitForGhostBricks( $\langle \mathbf{W} \rangle_{i+\frac{1}{2}\mathbf{e}^d}$ )
       $\langle \bar{\mathbf{F}} \rangle_{i+\frac{1}{2}\mathbf{e}^d} \leftarrow \text{FluxValues}(\langle \mathbf{W} \rangle_{i+\frac{1}{2}\mathbf{e}^d})$ 
       $\frac{d\langle \mathbf{U} \rangle_i}{dt} \leftarrow -\frac{1}{\Delta x} \sum_{d=1}^D (\langle \bar{\mathbf{F}} \rangle_{i+\frac{1}{2}\mathbf{e}^d} - \langle \bar{\mathbf{F}} \rangle_{i-\frac{1}{2}\mathbf{e}^d})$ 
    end for
    // Step 4: Update conservative variables
    for each Brick in Domain do
       $\langle \mathbf{U} \rangle_i^{s+1} \leftarrow \langle \mathbf{U} \rangle_i^s + \Delta t \cdot \frac{d\langle \mathbf{U} \rangle_i}{dt}$ 
      SendGhostBricks( $\langle \mathbf{U} \rangle_i^{s+1}$ )
    end for
  end for
  // Final update after RK4 stages are calculated
   $\langle \mathbf{U} \rangle_i^{n+1} \leftarrow \text{updateAfterRK4}()$ 
end for
  
```





Euler equations:

$$\frac{\partial \mathbf{U}}{\partial t} + \vec{\nabla} \cdot (\mathbf{F}) = \mathbf{S} \quad (1)$$

$$\mathbf{U} = \begin{bmatrix} \rho \\ \rho \mathbf{u} \\ \rho e \end{bmatrix}, \mathbf{F} = \begin{bmatrix} \rho \mathbf{u} \\ \rho \mathbf{u} \mathbf{u} + p \vec{I} \\ \rho \mathbf{u} \left(e + \frac{p}{\rho} \right) \end{bmatrix}, \mathbf{S} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad (2)$$

Closure relations:

- Ideal gas equation:

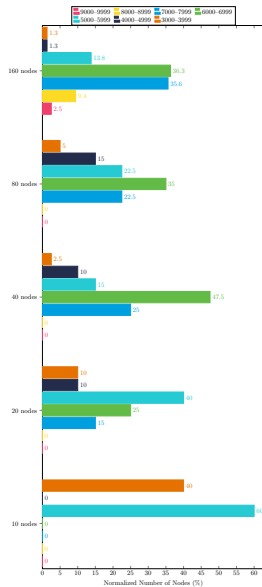
$$p = \rho R T \quad (3)$$

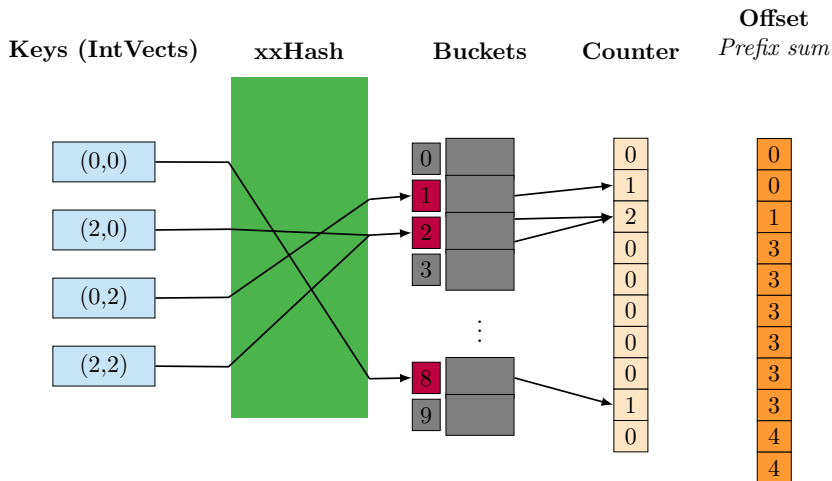
- Energy equation:

$$e = c_v T + \frac{1}{2} (\mathbf{u} \cdot \mathbf{u}) \quad (4)$$

Results

Ghost brick distribution





Keys (Int Vects) Buckets Offset HashGraph

